Lab 08 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — Three images for three environments

Answer. The image tested in staging (`-staging`) is **not** the one deployed to production (`-prod`): two builds, two contents, and the production configuration was exercised nowhere before going live. There should have been **a single image** `api:1.4.2`, built once, with configuration injected at deployment (`--env-file`, mounted file, secrets). You lose: the guarantee "what was tested is what runs", traceability by digest, the ability to promote the same image; and you gain three builds, three scans, three copies in storage.

Why. An image is an immutable artefact identified by its digest; that digest is what you promote. As soon as configuration is inside, the artefact changes with the environment.

Nuance. An `ENV` with a **default value** in the image (`APP_PROFILE=default`, `SERVER_PORT=8080`) is legitimate: it describes the image, not an environment.

Example.

```
podman run -d --name api-staging --env-file staging.env api:1.4.2
podman run -d --name api-prod    --env-file prod.env    api:1.4.2     # same image, same digest
```

Question 2 — Source precedence

Answer. `staging`, `prod`, `prod`, `shell`. The order of the options does not matter because Podman does not apply the sources "in line order" but by **category**: first the image's `ENV`, then the `--env-file` files, then the `-e` options, the later ones overriding the earlier. `-e APP_PROFILE` with no value copies the variable from the host shell.

Why. It is a precedence rule, not a sequence: the value "closest" to the command wins (command line > file > image).

Nuance. Inside an `--env-file`, the **last** line for a given key wins; and Docker applies the same `-e` > `--env-file` rule.

Example.

```
podman run --rm -e APP_PROFILE=prod --env-file api.env api:env env | grep APP_PROFILE   # prod
```

Question 3 — Port 9090 and `-p 18080:8080`

Answer. `SERVER_PORT=9090` made the application listen on **9090**; `-p 18080:8080` relays to 8080, where nobody listens any more: connection refused inside the container, `curl` gets no answer. `EXPOSE 8080` changes nothing: it describes the image, not the process's actual configuration. Align them: `-p 18080:9090`, or leave the port alone.

Why. Three independent layers: the application listens where its configuration says, `-p` relays where you tell it, `EXPOSE` documents. None of them checks the other two.

Nuance. On a user-defined network (lab 07), the other containers would also have to call `api:9090`. Changing a port through a variable reaches further than it seems.

Example.

```
podman run -d --name api2 -p 18080:9090 -e SERVER_PORT=9090 api:1.0 && sleep 2 && curl -s localhost:18080/actuator/health
```

Question 4 — Password as an argument

Answer. From most to least serious: (1) **the logs** — the application prints its arguments (`Arguments recus : --db-password=Secr3t`), and logs get copied, centralised, retained; (2) **the host's `ps`** — any user of the machine reads `ps -eo args`; (3) **`podman inspect`** (`Config.Cmd`) — anyone with access to the engine; (4) **`podman top api args`** — likewise. Add the shell history (`~/.bash_history`).

Why. A process's command line is public system data (`/proc/<pid>/cmdline`), and a container's command is stored in its configuration.

Nuance. Even if the application printed nothing, `ps` would be enough. An argument is never a channel for a secret, inside or outside a container.

Example.

```
podman logs api | head -1 ; podman inspect api --format '{{json .Config.Cmd}}' ; ps -eo args | grep -c 'db-password=Secr3t'
```

Question 5 — Password as a variable

Answer. Yes, it is better: it vanishes from the argument logs, from `ps` and from `Config.Cmd`. It remains visible in `podman inspect` (`Config.Env`), `podman exec api env`, `/proc/1/environ` inside the container, and in any error report that dumps the environment. With `POSTGRES_PASSWORD_FILE=/run/secrets/db_password`, `Config.Env` holds only that **path**: the value is readable only by a process in the container that opens the file.

Why. The environment is container metadata, stored by the engine; a mounted file is not.

Nuance. `podman exec db cat /run/secrets/db_password` remains possible for anyone with engine access. The gain is removing the value from the **metadata** and the exports (`inspect`, Compose, tickets), not making it unreachable for the administrator.

Example.

```
podman inspect db --format '{{range .Config.Env}}{{println .}}{{end}}' | grep POSTGRES   # POSTGRES_PASSWORD_FILE=/run/secrets/db_password
```

Question 6 — Spring Boot and the environment

Answer. *Relaxed binding* converts a variable name into a property name: upper case → lower case, `_` → `.` (`SPRING_DATASOURCE_URL` → `spring.datasource.url`). `SPRING_PROFILES_ACTIVE=prod` activates the `prod` profile: Spring Boot additionally loads `application-prod.yaml` and the beans annotated `@Profile("prod")`. On conflict, the **environment variable** beats

`/config/application.yaml` (external file, outside the JAR), which beats `application.yaml` inside the JAR.

Why. Spring Boot orders its `PropertySources` from the most external to the most internal, precisely to allow injection without a rebuild.

Nuance. `--spring.datasource.url=…` arguments come before variables — but they are arguments, hence visible (question 4). And `spring.config.import=optional:configtree:/run/secrets/` turns every file in `/run/secrets/` into a property.

Example.

```
podman run -d --name api -e SPRING_PROFILES_ACTIVE=prod -e
SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/app -v
./application.yaml:/config/application.yaml:ro real-api:1.0
```

Question 7 — Questionable Dockerfile

Answer. `ENV SPRING_PROFILES_ACTIVE=prod`: freezes an environment into the image — pass it with `-e` at deployment. `ENV SPRING_DATASOURCE_PASSWORD=Pr0d!`: a secret in the image, readable through `image inspect` and `history` — pass it through a secret (`_FILE` / `configtree`). `COPY .env /app/.env`: a local configuration file, probably with secrets, in a layer — put it in `.dockerignore`, and supply it at `run` through `--env-file` or a mount. Corrected:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/api.jar /app/api.jar
USER 1000:1000
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/api.jar"]
```

Why. The image must contain only what is true everywhere: the code and its defaults. Everything else is injected.

Nuance. An `ENV SPRING_PROFILES_ACTIVE=default` or `ENV JAVA_TOOL_OPTIONS=-XX:MaxRAMPercentage=75` would remain acceptable: defaults, not environments.

Example.

```
podman image inspect api --format '{{range .Config.Env}}{{println .}}{{end}}' | grep -i -E
'password|profile'   # must be empty
```

Question 8 — `podman secret` in practice

Answer. `Config.Env` contains `POSTGRES_PASSWORD_FILE=/run/secrets/db_password` and nothing else about the password; `ls -l /run/secrets/` shows `-r--r--r-- 1 root root 6 … db_password`; `podman secret inspect db_password` gives the ID, the dates, the `file` driver and its path, **not the value**. `--showsecret` prints it: proof that the storage is a drawer readable by your user, not encryption.

Why. The secret is copied into the container at start-up as a file; the engine puts it neither in the environment nor in the container's configuration.

Nuance. The file belongs to `root` and is readable by everyone inside the container (`444`): a non-root process reads it. `--secret db_password,mode=0400,uid=70` tightens the permissions.

Example.

```
podman secret inspect --showsecret db_password --format '{{.SecretData}}'   # S3cr3t
podman inspect db --format '{{json .Config.Env}}' | grep -c 'S3cr3t'          # 0
```

Question 9 — "You might as well use `-e`"

Answer. Compared with `-e`, `podman secret` removes the value from `podman inspect`, from the process environment (in file mode), from Compose files and from commands in the shell history; it creates the value **once** and references it by name everywhere; and it mounts it with controllable permissions. What it does not bring: encryption at rest, access control, a log of reads, rotation. That is exactly what a secrets manager adds: who read what and when, encryption under a master key, automatic renewal and short-lived tokens.

Why. `podman secret` solves the problem of **accidental exposure**; a manager solves the problem of **governance**.

Nuance. On a development workstation, `podman secret` is more than enough. The `pass` driver (GPG) exists to encrypt locally.

Example.

```
podman secret ls ; podman secret inspect db_password --format '{{.Spec.Driver.Name}}'   # file
```

Question 10 — The build succeeds without the secret

Answer. The second build found all its instructions **in cache**, including the `RUN --mount=type=secret`, which was therefore not re-executed — and a `RUN` that does not run asks for no secret. To see it fail: `podman build --no-cache`, or change an earlier instruction. Consequence: an expired token goes undetected as long as the cache holds; on a CI agent with a persistent cache, the build "works" until the day a dependency changes.

Why. The cache is keyed on the instruction and the context, not on the secrets supplied: supplying a secret or not does not invalidate it.

Nuance. It is also a protection: since the secret is not in the cache key, its value cannot be derived from the cache.

Example.

```
podman build --no-cache -t app .   # Error: … secret required but no secret with id "token" found
```

Question 11 — `ARG` versus `--mount=type=secret`

Answer. With `ARG NPM_TOKEN`, the token sits in the layer's metadata: `podman history --no-trunc` shows `|1 NPM_TOKEN=… /bin/sh -c npm ci`. With the secret mount, it is nowhere: `history` shows the line `--mount=type=secret,id=npm_token npm ci`, with no value, and `/run/secrets` does not exist in the image. Multi-stage protects the `ARG` **partially**: if the `ARG` is declared in a discarded

stage, the final image has no history of it. But an `ARG` in the final stage stays visible, and the build-stage image (`--target`) keeps everything.

Why. `ARG` is substituted into the recorded command of the layer; a mount is not recorded.

Nuance. Buildah and BuildKit treat both the same way. The only difference: Buildah ignores `# syntax=` and does not need it.

Example.

```
podman history --no-trunc app --format '{{.CreatedBy}}' | grep -i token
```

Question 12 — `.env` in Git

Answer. (1) **Git does not forget**: even deleted in the next commit, the file stays in the history, in every clone and every fork. (2) **Development is not isolated**: dev passwords often open shared services (test database, registry, third-party API in sandbox mode), and leak scanners (Gitleaks, GitHub secret scanning) will flag it. (3) **Habit**: the "it is only dev" reflex ends up committing a staging value. Practice: `.env` in `.gitignore` and `.dockerignore`, a committed `.env.example` with the keys and empty values, and for CI, secrets injected by the tool.

Why. A repository is a distribution medium; everything that enters it must be able to become public.

Nuance. A committed `.env` **with no secret at all** (ports, service names) is acceptable, and common with Compose (lab 09). The problem is the content, not the file.

Example.

```
git log --all -p -- .env | grep -i password   # what the history keeps
```

Question 13 — Leak in a ticket

Answer. (1) Treat the secret as **compromised**: the question "who saw it" has no reliable answer. (2) **Rotate** the secret: new password on the database side, new value in the secrets manager or `podman secret`, redeploy the consumers. (3) **Revoke** the old one, check the database access logs for the period. (4) Clean the ticket, **afterwards**, and address the cause: why was a full `inspect` needed, why did it contain a secret (hence `-e` instead of a file). Deleting the comment does nothing: it has been read, emailed in notifications, maybe indexed. Speed depends on the architecture: secret referenced by name and injected at start-up → a few minutes; secret hand-copied into configuration files on servers → hours and omissions.

Why. The only safe answer to a leak is to make the leaked value **useless**.

Nuance. That is the decisive argument for automatic rotation: if changing a secret is a daily, painless operation, a leak is a minor incident.

Example.

```
podman secret rm db_password && printf 'new' | podman secret create db_password - && podman restart api db
```